



CIRCUITOS DIGITALES Y MICROCONTROLADORES 2025

Facultad de Ingeniería
UNLP

Máquinas de Estados Finitos
y su implementación en Software

Ing. José Juárez

Modelización con Máquinas de Estados

- **Abstracción:** Se trata de definir un problema complejo en un conjunto de principios básicos. Si podemos modelar el sistema en base a estos principios, podremos entender mejor el problema, separando “que es lo que se está haciendo” de los detalles del “como se esta haciendo”.
- **Definición 1:** Una una Máquina de Estados Finitos (MEF) es un modelo abstracto del “comportamiento” del sistema, basado en principio simples.
- **Definición 2:** una Máquina de Estados Finitos (MEF) es un modelo matemático (Teoría general de autómatas) usado para describir el comportamiento de un sistemas que puede ser representado por un número finito de estados, un conjunto de entradas y una función de transición que determina el estado siguiente en función del estado actual y de las entradas.
- **Ventajas de la implementación con “maquina de estados”**
 - Hace más sencilla la comprensión del sistema y su funcionamiento,
 - Desde el punto de vista del software es más sencillo de mantener ya que se pueden agregar o quitar estados sin modificar el resto
 - Es más sencillo de depurar,
 - Es más sencillo de verificar
 - Es más sencillo de Optimizar

[Wikipedia: Teoría de Autómatas](#)

[Wikipedia: Autómata finito](#)

Modelización con Máquinas de Estados

- **El modelo :**

- Contiene los estados, las entradas y las reglas bien definidas para cambiar de estado
- El siguiente estado depende de las entradas y del estado actual
- Se puede especificar mediante un “**diagrama de estados**” o “**tabla de transiciones de estados**”
- Cada transición implica diferentes respuestas o acciones del sistema
=> 2 posibilidades para actualizar las salidas

- **Mealy**

- La salida depende del estado actual y de las entradas
- Son propensos a este modelo, los sistemas donde la salida provoca el cambio de estado. Por ejemplo, el movimiento de las articulaciones de un robot producen el cambio de estado (parado-sentado)

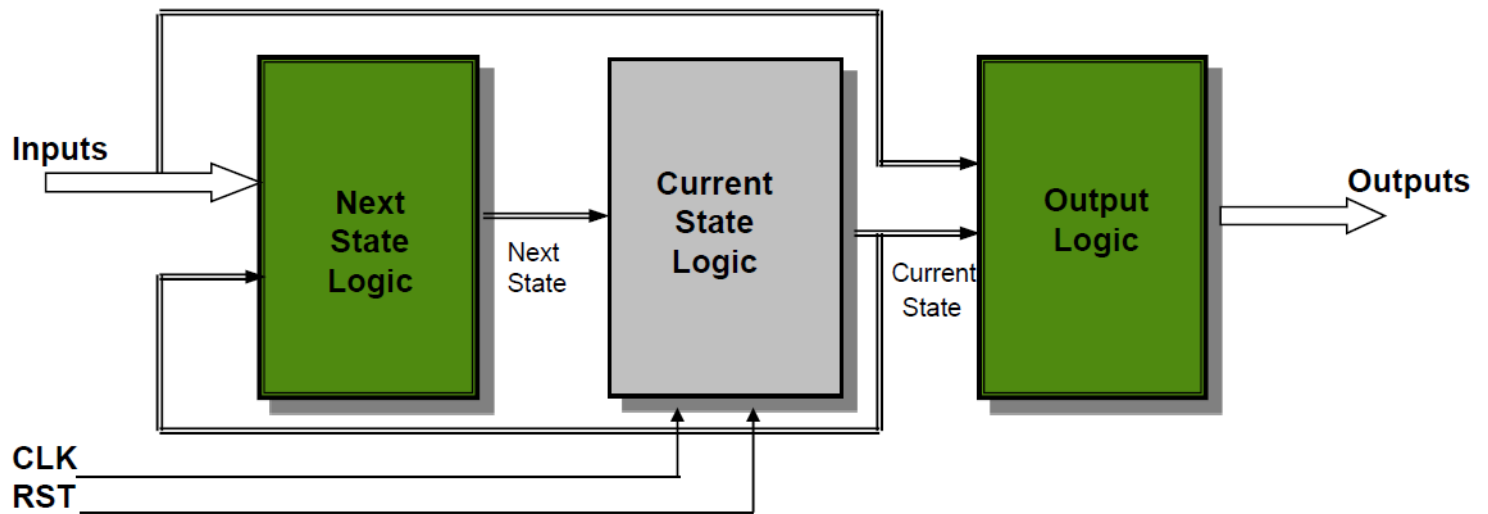
- **Moore**

- La salida del sistema depende solo del estado actual
- Puede haber múltiples estados con la misma salida, pero para cada estado el significado es diferente. La salida guarda estrecha relación con el estado. Por ejemplo, un controlador de Semáforo

- Ambos modelos son intercambiables pero es mejor optar por la forma que representa de manera más natural el problema.

Implementación: Hardware

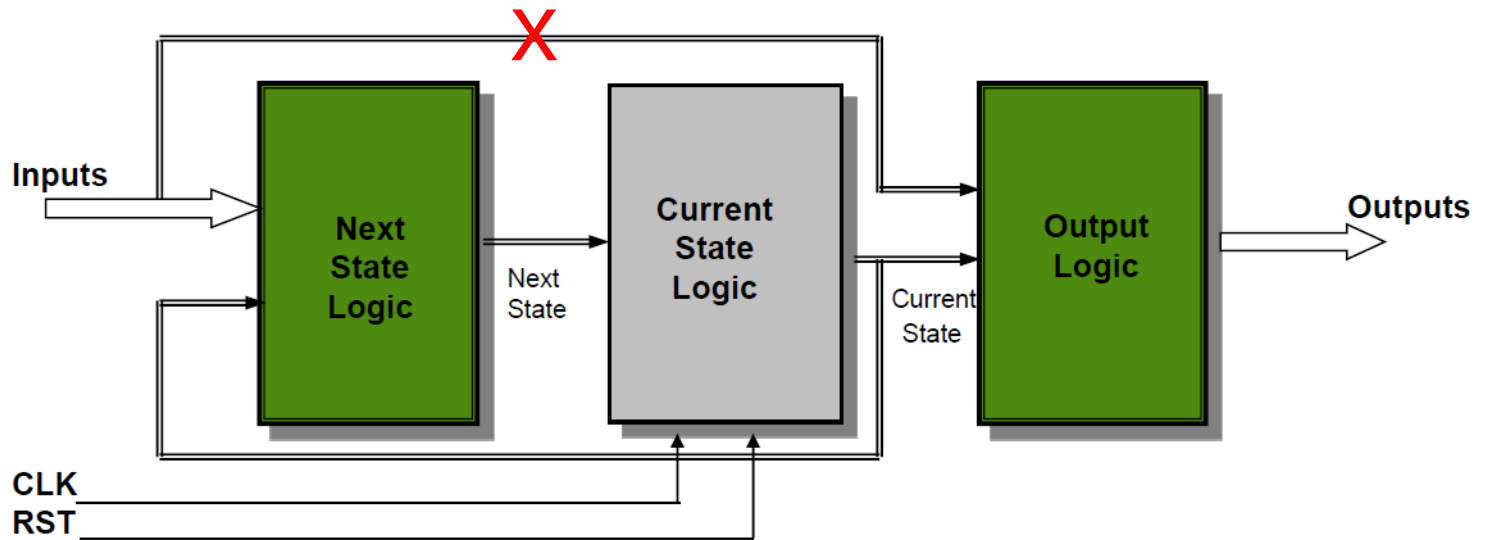
- Modelo General de un circuito secuencial síncrono:
(Modelo de Mealy)



Implementación: Hardware

- Modelo General de un circuito secuencial síncrono:
(Modelo de Mealy)

Moore



Especificación

- 1-Descripción matemática (6-tupla)

$$MEF = \{S, s_0, S_f, \Sigma, \delta, \Gamma\}$$

S : Conjunto Finito de estados del sistema

$s_0 \in S$: Estado inicial

$S_f \subseteq S$: conjunto de estados finales

Σ : alfabeto finito de entradas o eventos que producen cambios de estado

δ : función de transición $S \times \Sigma \rightarrow S$

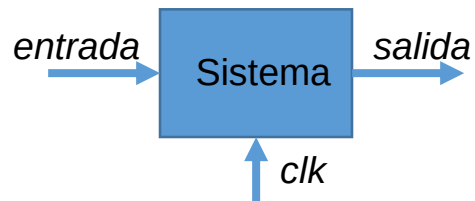
Γ : Conjunto finito de salidas

Especificación

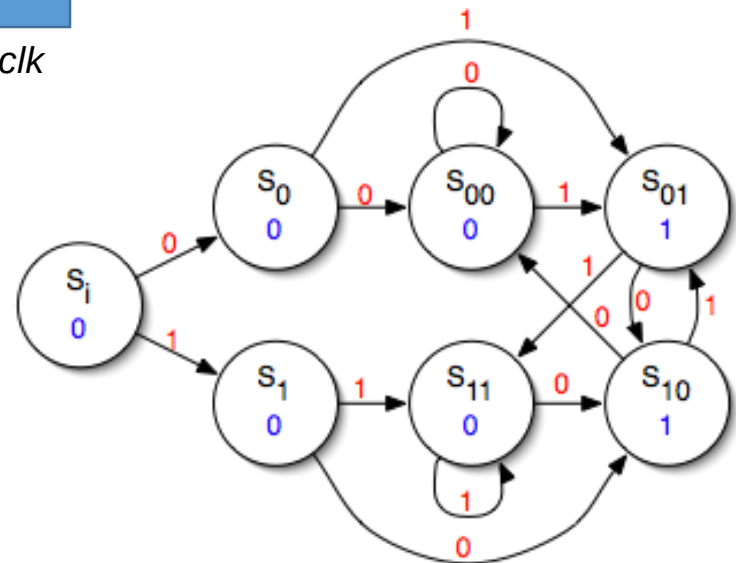
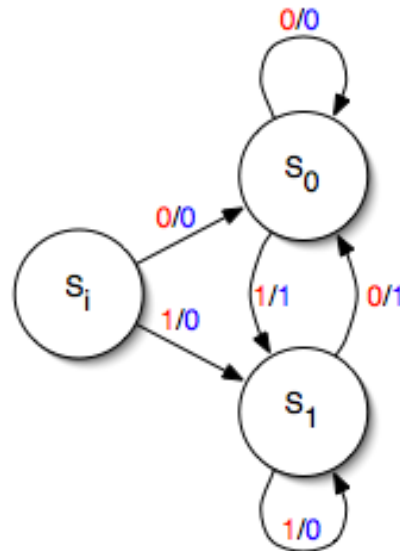
- 2-Descripción por diagramas de estados

- Ejemplo: Detector de flanco - la salida es la XOR de las dos entradas mas recientes

- **Moore:**



- **Mealy:**



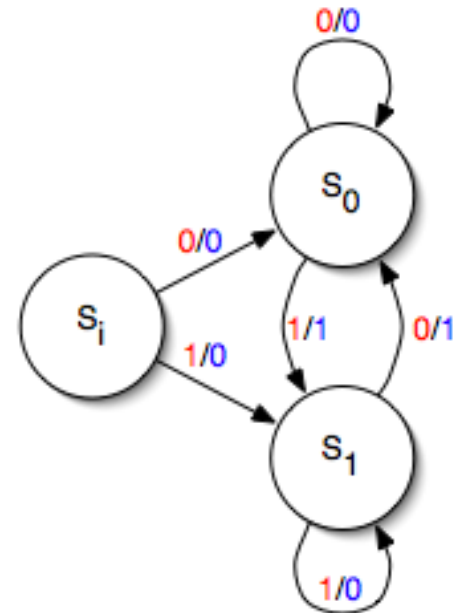
En rojo: entrada

En azul: salida

Especificación

- 3-Descripción por tabla de transición de estados

Estados\Entradas	1	0
S _i	S ₁ /0	S ₀ /0
S ₀	S ₁ /1	S ₀ /0
S ₁	S ₁ /0	S ₀ /1



Implementación: MEF en software

- Del análisis de la especificación podemos extraer los siguientes elementos a implementar:

1) Definir el conjunto de estados

estados: (S0, S1,...,Sn)

2) Definir el conjunto de entradas (alfabeto)

entradas: (i0, i1,...,im)

3) Definir el conjunto de salidas

salidas: (O0, O1,...,Ok)

4) Definir una función de transición de estados

tabla : [1...n] [1...m]

5) Definir procedimiento para establecer el estado inicial

```
Iniciar_MEF {  
    estado=Si    //i e 0..n  
    salida=Oj    //j e 0..k  
}
```

Implementación: MEF en software

6) Definir procedimiento para actualizar la MEF

```
Actualizar_MEF() {  
    Leer(&entradas);  
    estado=tabla[estado][entradas];  
    Act_Salidas(estado,entradas);  
}
```

Implementación
por tablas



7) Definir procedimientos para ejecutar la MEF MEF *(es independiente del modelo y depende de la arquitectura del software: time-triggered o event – triggered)*

Sin temporizar o
periódica cada T



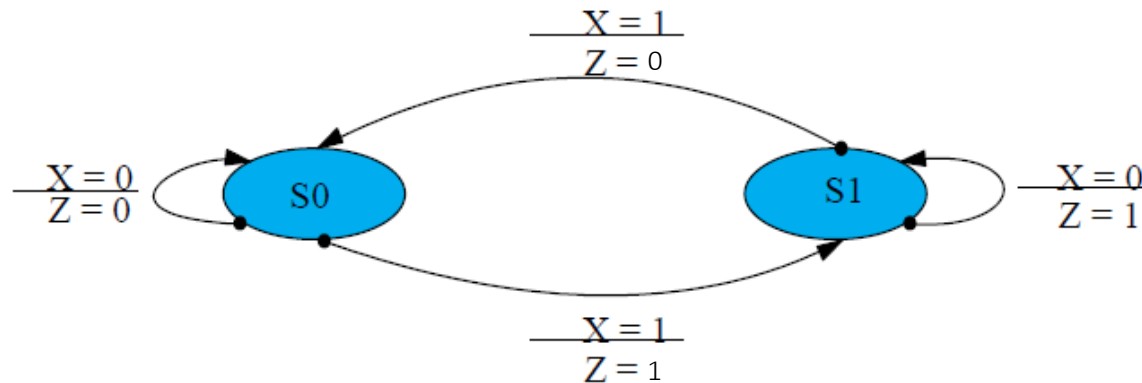
```
Ejecutar_MEF_x{  
    Iniciar_MEF_x();  
  
    repetir siempre o cada T [seg]{  
        Actualizar_MEF_x();  
    }  
}
```

Puede ser
Bloqueante o
No-bloqueante



Implementación: switch-case

- Implementación a partir de la especificación:
si examinamos un diagrama de estados, por ejemplo :



surge intuitivamente el siguiente razonamiento:

Estado S0:

```
If (X==0)  
  then S0 y Z=0  
else S1 y Z=1
```

Estado S1:

```
If (X==0)  
  then S1 y Z=1  
else S0 y Z=0
```

Notar que si

fuera Moore

Las salidas van

fuera del if-else

¿cómo seleccionar entre S0 o S1? => switch- case

Implementación: switch-case

```
void ActualizarMEF(void)
{
    X=leerEntradas();
    switch(estado) {
        Case S0:
            if(X==1)
                { estado=S1; Z=1;}
            else
                { estado=S0; Z=0;}
            break;
        Case S1:
            if(X==1)
                { estado=S0; Z=0;}
            else
                { estado=S1; Z=1;}
            break;
    }
}
```

Si hay múltiples
entradas => se
puede poner otro
switch-case en
lugar de if-else



Implementación: switch-case

*Definición y
Declaración de
Variables de estado*



```
typedef enum{S0,S1}state;  
state estado;
```

Método de Inicialización →

```
Iniciar_MEF() {  
    estado=S0; Z=0;  
}
```

Ejecución de la MEF →



```
void main(void) {  
    Iniciar_MEF();  
  
    while(1) {  
        //se puede repetir cada T  
        //delay(T);  
        Actualizar_MEF();  
    }  
}
```

Implementación: con punteros a función

```
typedef enum{S0,S1} state;

void fS0(void);
void fS1(void);
void (*MEF[]) (void)={fS0,fS1};
state estado;

void main(void) {
    Iniciar_MEF();
    while(1){
        Actualizar_MEF();
    }
}

void ActualizarMEF(void) {
    X=leerEntradas();
    (*MEF[estado]) ();
}

void fS0(void)
{
    if(X==1) {
        estado=S1; Z=1;}
    else {
        estado=S0; Z=0;}
}

void fS1(void)
{
    if(X==1) {
        estado=S0; Z=0;}
    else {
        estado=S1; Z=1;}
}

//Ejecuta la función correspondiente
```

Implementación: con punteros a función

- ***Diferencias:***

- En implementaciones con **switch()** los **case** de cada estado se evaluarán secuencialmente, equivale a una cadena de **if** consecutivos de resolución. No tarda lo mismo en ejecutar las actualizaciones según el caso.
- En implementaciones con **punteros a función o tablas** el tiempo de acceso a las funciones es el mismo independientemente del valor de la variable de estado, equivale a un desvío selectivo de la ejecución del programa.
- En general implementaciones con punteros o tablas de transición permiten uniformidad en el tiempo de acceso, son más compactas, pero ocupan más memoria.

- ***Otros tipos de implementaciones:***

- Tabla de transición, listas enlazadas y otras.

Implementación: MEF en Software

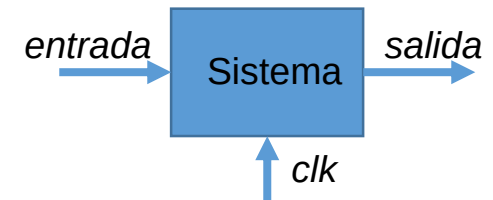
- **MEF Temporizadas:** Actualización periódica con Timer (polling periódico)

```
typedef enum={S0,S1}state;  
state estado;
```

```
void main(void) {  
    Iniciar_MEF();
```

```
    while(1) {  
        if (FLAG_TIMER) {  
            Actualizar_MEF();  
            FLAG_TIMER=0;  
        }  
        sleep();  
    }
```

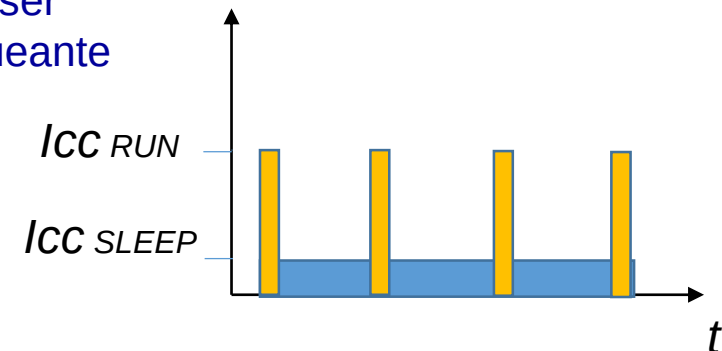
Similar a la implementación en Hardware



Flag modificado x ISR de Timer

Debe ser
No-bloqueante

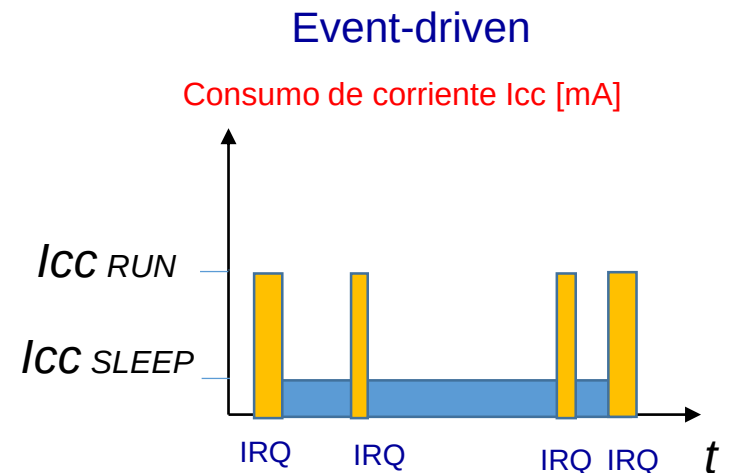
Consumo de corriente I_{CC} [mA]



Implementación: MEF en Software

- **MEF controladas por eventos:** Actualización por eventos no periódicos generalmente (puede derivar en una arquitectura con múltiples interrupciones)

```
Ejecutar_MEF{  
  
    Iniciar_MEF();  
    repetir siempre{  
        sleep();  
        Actualizar_MEF();  
    }  
}  
  
ISR(IRQ_request) {  
    Leerentradas();  
}
```



Implementación: MEF en Software

- **MEF jerárquicas:** Un estado puede contener otra MEF

```
void ActualizarMEF(void)
{
    switch (estado) {
        Case S0:
            Actualizar_subMEF_S0(&params)
            break;
        Case S1:
            if (X==1)
                { estado=S0; Z=0; }
            else
                { estado=S1; Z=1; }
            break;
    }
}
```

Tiene que heredar todas las variables del estado superior

Implementación: MEF en Software

- **MEF cooperativas** varias MEF independientes en ejecución “simultánea”

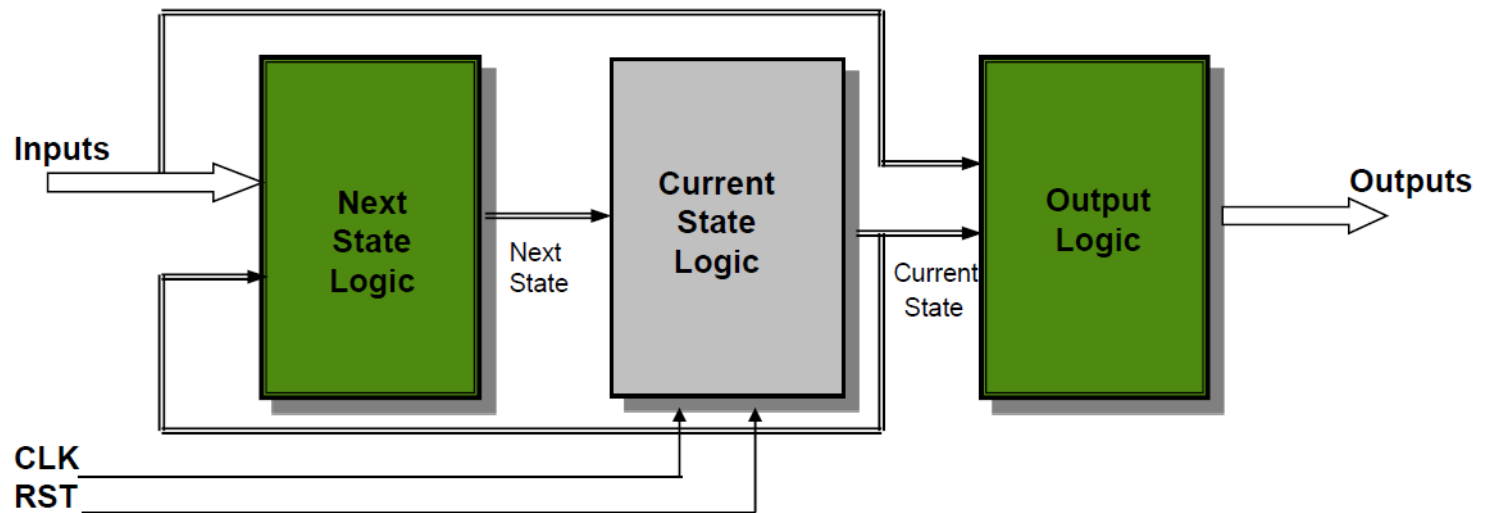
```
void main(void) {  
    Iniciar_MEF_1();  
    Iniciar_MEF_2();  
  
    while(1) {  
        Actualizar_MEF_1();  
        Actualizar_MEF_2();  
    }  
}
```

- 1-No-bloqueantes
- 2-Independientes (2 diagramas)
- 3-Se comunican x mensajes
asincrónicos
- 4-Se ejecutan hasta completar
(run to completion)
- 5-Son cooperativas (corrutinas)

- **Statecharts:** Método gráfico (especificación UML) para especificar MEFs generalizadas (jerárquicas y concurrentes)

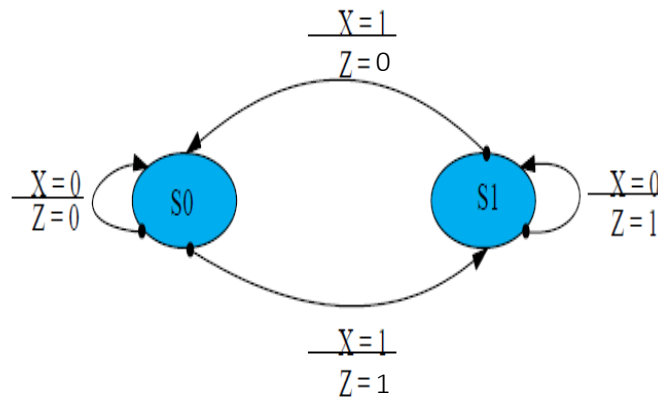
Implementación: MEF en Hardware

- Volvamos al modelo general de un circuito secuencial síncrono:
(Modelo de Mealy)
- Veamos como se implementaría el ejemplo anterior



Implementación: MEF en FPGA

Partimos de la misma especificación:



La MEF se implementa en un circuito
Secuencial descrito
En VHDL utilizamos las sentencias Process y
Case-When

```
type state is (S0,S1);  
...  
process(clk, rst)  
begin  
    if(rst= '1')then  
        state <= S0;  
        Z <= '0';  
    elsif(rising_edge(clk)) then  
        case state is  
            when S0 =>  
                if(X='0') then  
                    state <= S0;  
                    Z <= '0' ;  
                elsif(X='1') then  
                    state <= S1 ;  
                    Z <= '1';  
                end if;  
            when S1 =>  
                if (X = '0') then. . .  
                end if;  
            end case;  
        end if;  
    end process;
```

Implementación: comparación sintáctica

```
typedef enum{S0,S1} state;
state estado;
```

En C :

```
Iniciar_MEF(){
    estado=S0; Z=0;
}

void ActualizarMEF(void)
{
    switch(estado){
        Case S0:
            if(X==1)
                { estado=S1; Z=1;}
            else
                { estado=S0; Z=0;}
            break;
        Case S1:
            if(X==1)
                { estado=S0; Z=0;}
            else
                { estado=S1; Z=1;}
            break;
    }
}
```

```
type state is (S0,S1);
```

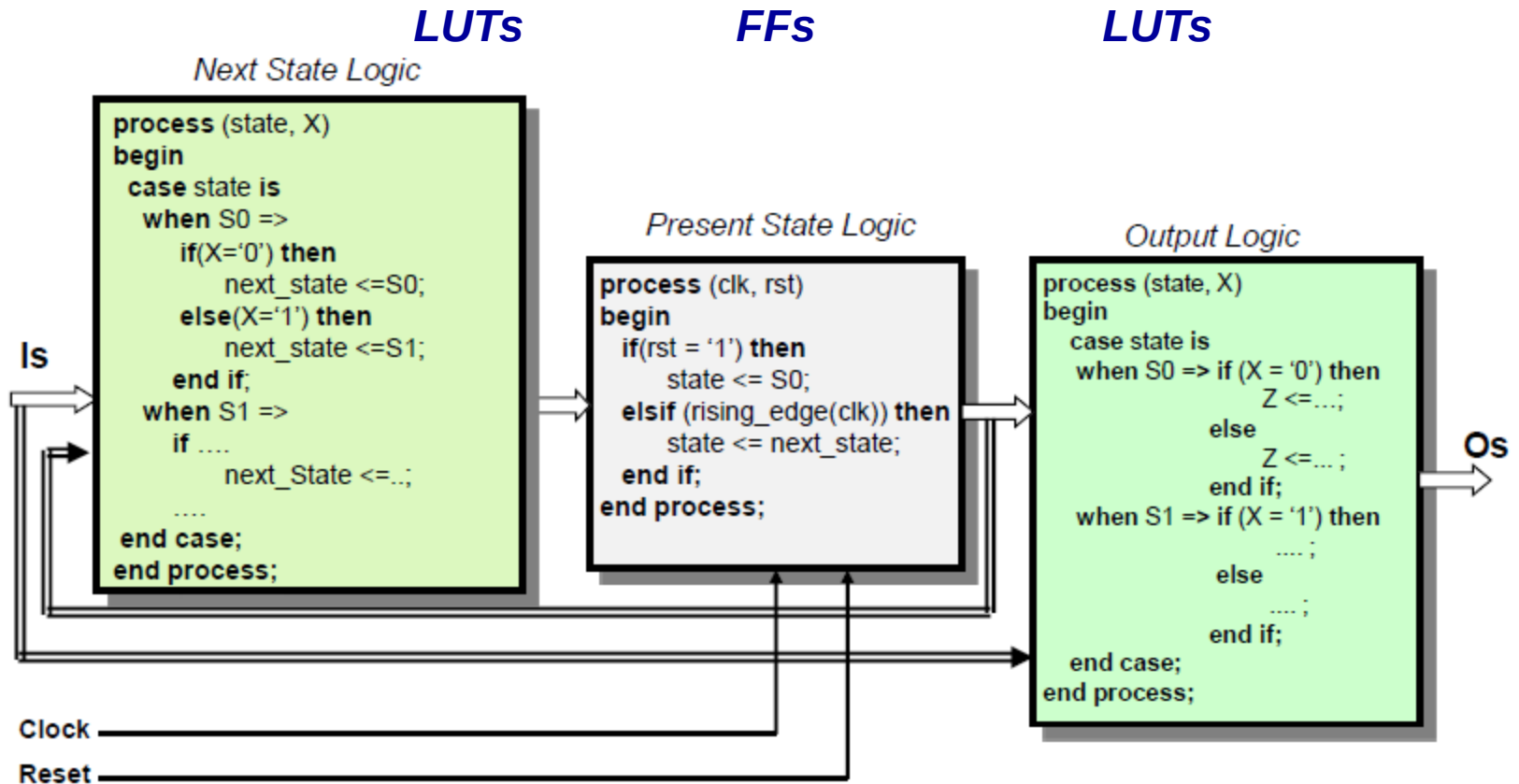
```
...
```

En VHDL :

```
process(clk, rst)
begin
    if(rst= '1')then
        state <= S0;
        Z <= '0';
    elsif(rising_edge(clk)) then
        case state is
            when S0 =>
                if(X='0') then
                    state <= S0;
                    Z <= '0' ;
                elsif(X='1') then
                    state <= S1 ;
                    Z <= '1';
                end if;
            when S1 =>
                if (X = '0') then. . .
                    end if;
                end case;
            end if;
        end process;
```

Implementación: MEF en FPGA

- Método formal en VHDL:



Ejemplo 1

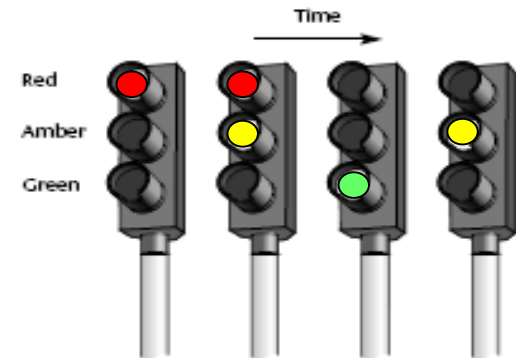
- Semáforo (MEF temporizada)

Estados y salidas (Moore):

- 1-rojo
- 2-rojo-amarillo
- 3-verde
- 4-amarillo

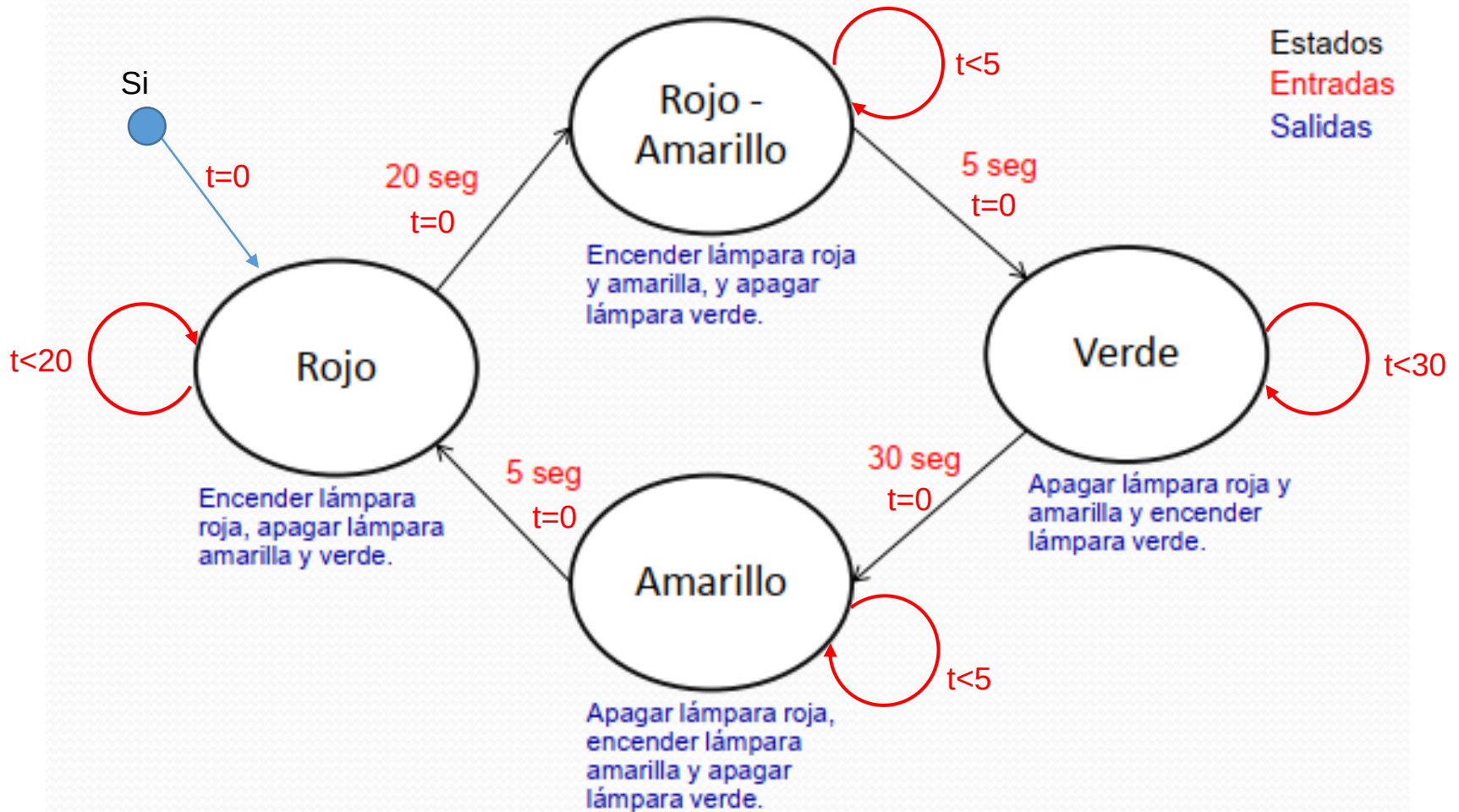
Cambio de estados x evento tiempo (entrada)

- Rojo: 20seg
- Rojo-amarillo: 5seg
- verde: 30seg
- Amarillo: 5 seg



Semáforo: Implementación MEF

Diagrama de estados del ejemplo de semáforo:



Semáforo: Implementación MEF

```
// Posibles estados del sistema
typedef enum {RED, RED_AMBER, GREEN, AMBER} eLight_State;

// Tiempo de duración de cada estado
// (tiempo en segundos)
//
#define RED_DURATION 20
#define RED_AMBER_DURATION 5
#define GREEN_DURATION 30
#define AMBER_DURATION 5
```

Semáforo: Implementación MEF

```
// ----- variables privadas-----

static eLight_State Light_state; // Estado actual del sistema
static long Time_in_state; // Tiempo en el estado

/*-----*-
TRAFFIC_LIGHTS_Init()
-*-----*/

void TRAFFIC_LIGHTS_Init ( eLight_State START_STATE)
{
    Light_state = START_STATE; // poner un estado inicial
}

void main(void)
{
    MCU_Init();

    TRAFFIC_LIGHTS_Init (RED); // inicializar MEF

    while(1){
        TRAFFIC_LIGHTS_Update(); // MEF temporizada aprox. c/1 seg
        delay_ms(1000);
    }
}
```

Semáforo Implementación MEF

```
void TRAFFIC_LIGHTS_Update(void)
```

```
{
```

```
switch (Light_state)
```

```
{
```

```
case RED:
```

```
Red_light = ON;
```

← *salidas*

```
Amber_light = OFF;
```

```
Green_light = OFF;
```

← *entrada*

```
if(++Time_in_state == RED_DURATION)
```

```
{
```

```
Light_state = RED_AMBER;
```

← *Cambio estado*

```
Time_in_state = 0;
```

```
}
```

```
break;
```

```
case RED_AMBER:
```

```
Red_light = ON;
```

```
Amber_light = ON;
```

```
Green_light = OFF;
```

```
if(++Time_in_state == RED_AMBER_DURATION)
```

```
{
```

```
Light_state = GREEN;
```

```
Time_in_state = 0;
```

```
}
```

```
break;
```

(Moore)

```
case GREEN:
```

```
Red_light = OFF;
```

```
Amber_light = OFF;
```

```
Green_light = ON;
```

```
if(++Time_in_state == GREEN_DURATION)
```

```
{
```

```
Light_state = AMBER;
```

```
Time_in_state = 0;
```

```
}
```

```
break;
```

```
case AMBER:
```

```
Red_light = OFF;
```

```
Amber_light = ON;
```

```
Green_light = OFF;
```

```
if(++Time_in_state == AMBER_DURATION)
```

```
{
```

```
Light_state = RED;
```

```
Time_in_state = 0;
```

```
}
```

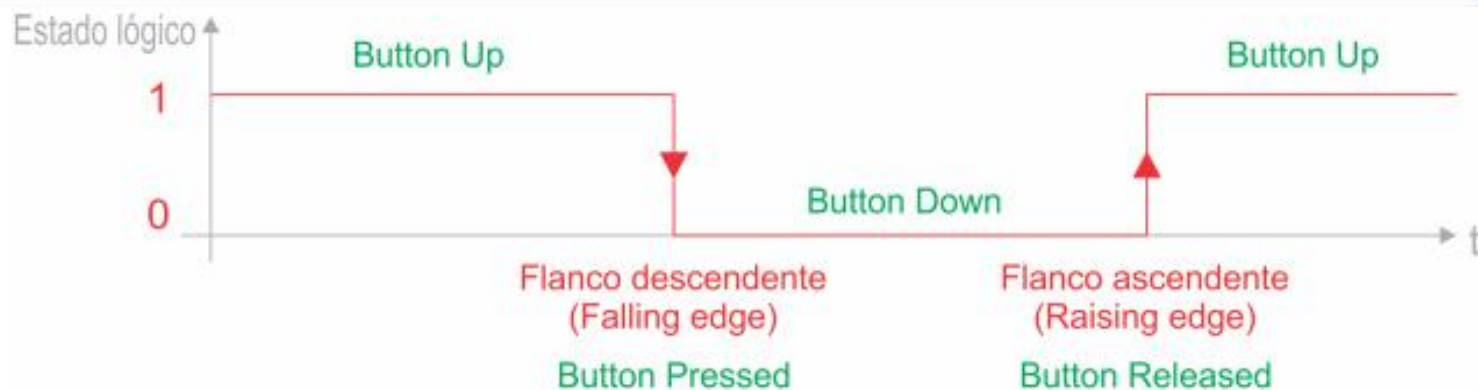
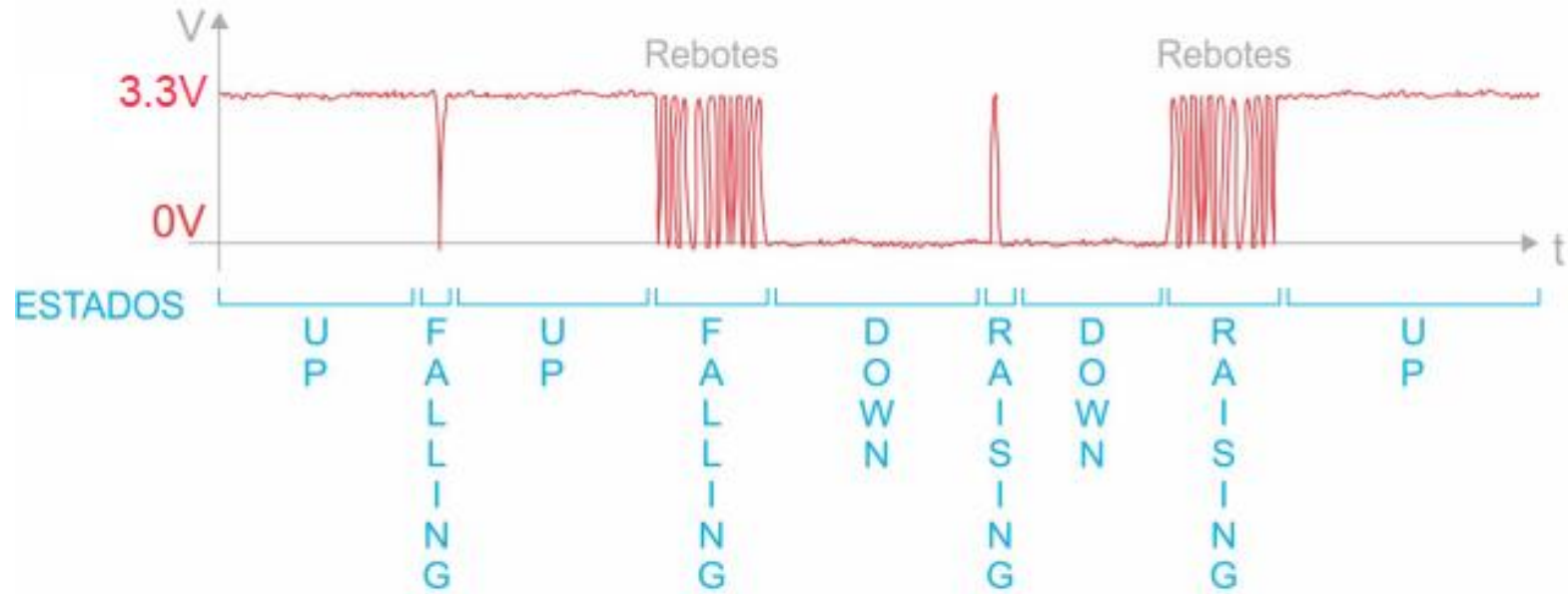
```
break;
```

```
}
```

```
}
```

Ejemplo 2

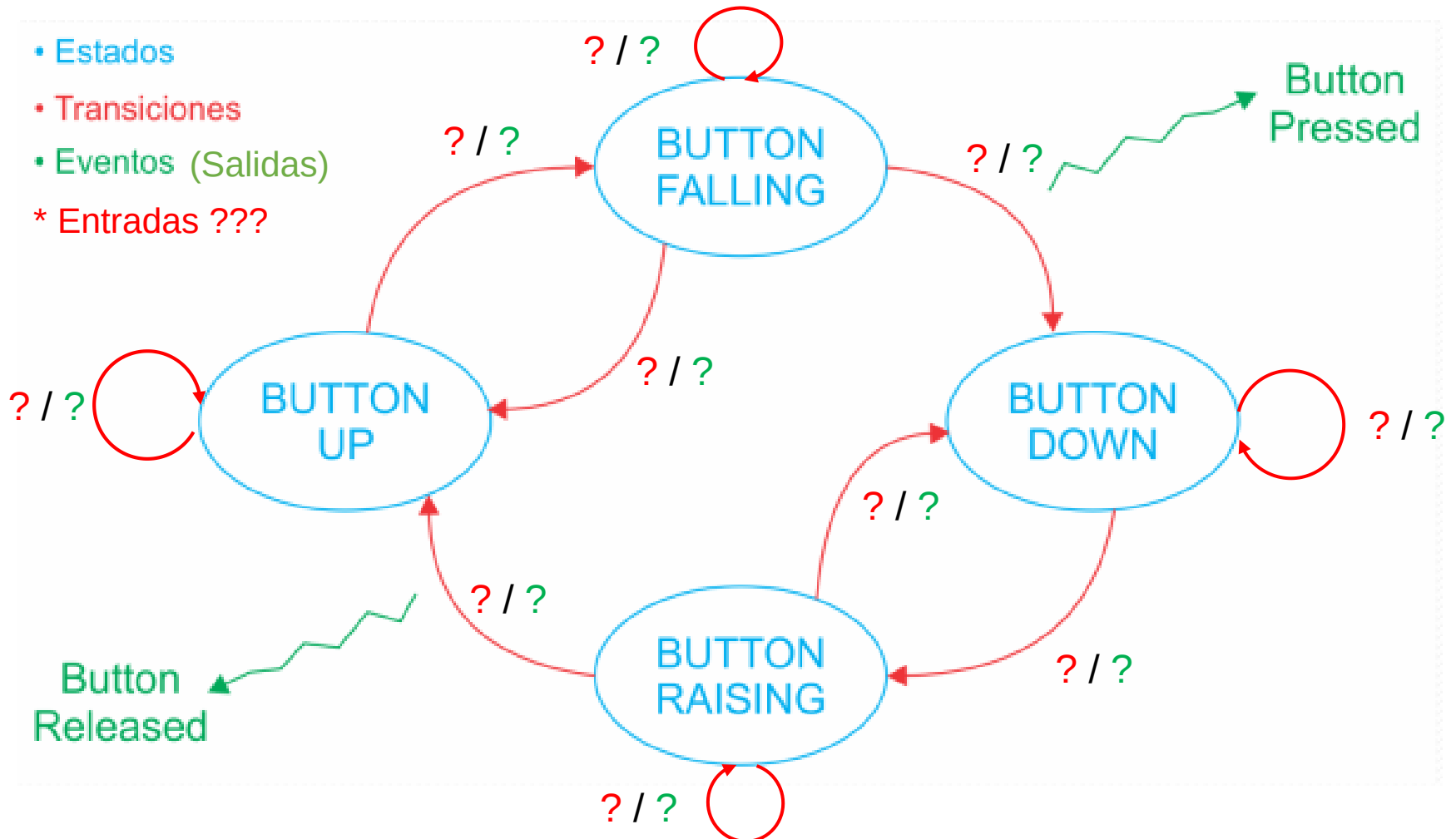
- Detección de pulsador con anti-rebote



Ejemplo 2

- Implementación MEF (Mealy)

Tarea: Completar especificación



Ejemplo 2: Detección de pulsador con anti-rebote

- En el contexto de una aplicación MEF que requiere como entrada pulsadores:

```
Actualizar_MEF_1() {  
    Leer(&entrada);  
    estado=tabla[estado][entrada];  
    Act_Salidas(estados, entrada);  
}
```

- El algoritmo anti-rebote sería una sub_MEF :

```
Actualizar_MEF_1() {  
    Actualizar_MEF_2(&entrada);  
    estado=tabla[estado][entrada];  
    Act_Salidas(estados, entrada);  
}
```

Devuelve Press o
Released



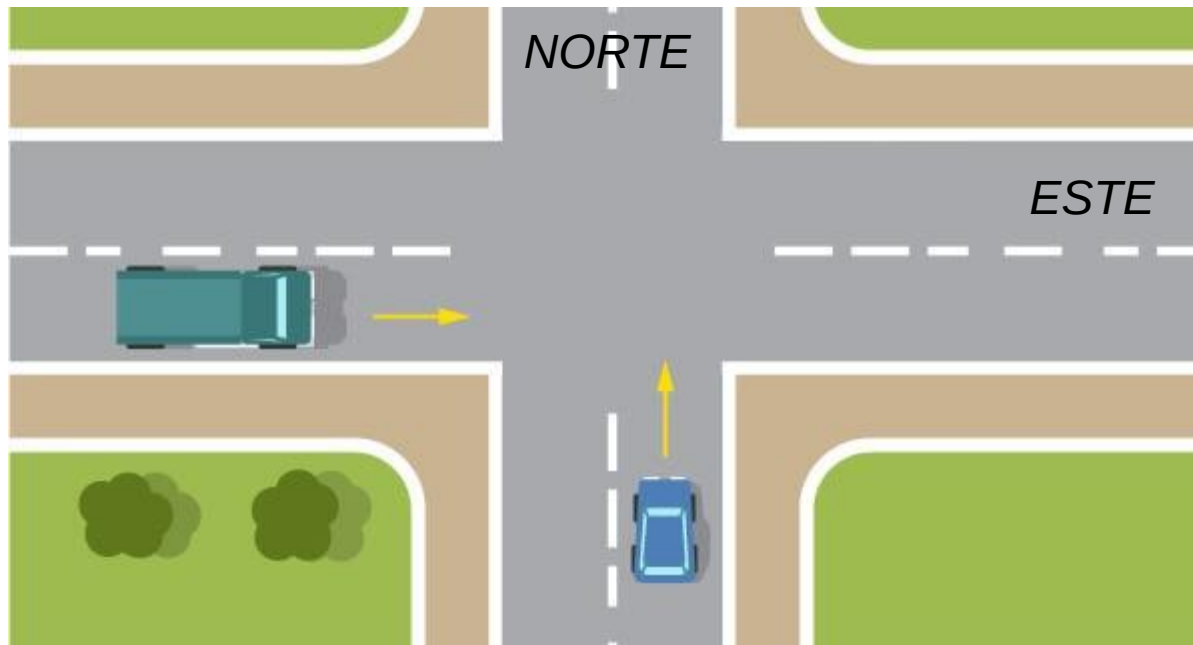
Ejemplo 3

- Retardo No Bloqueante con MEF
 - Otra solución al problema del retardo de visualización en el TP1

```
while(1){  
    if(FLAG_TIMER){  
        Actualizar_secuencia();  
        FLAG_TIMER=0;  
    }  
    otraTarea();  
}
```

```
void Actualizar_secuencia(void)  
{  
    switch (estado)  
    {  
        case PAR:  
            Encender_Leds(pares);  
            if(++Tiempo == DELAY_PARES)  
            {  
                estado = IMPAR;  
                Tiempo = 0;  
            }  
            break;  
  
        case IMPAR:  
            Encender_Leds(impares);  
            if(++Tiempo == DELAY_IMPARES)  
            {  
                estado = PAR;  
                Tiempo = 0;  
            }  
            break;  
    }  
}
```

Ejemplo4: Control de tráfico



Requerimientos:

- 1-Si no hay autos permanece en verde
- 2-Para cambiar de verde a rojo pasar por amarillo 5seg
- 3-El verde debe durar al menos 30seg
- 4-Si hay tráfico en una sola dirección el verde queda en esa dirección
- 5-Si el trafico es en ambas direcciones rotamos el verde

Entradas:

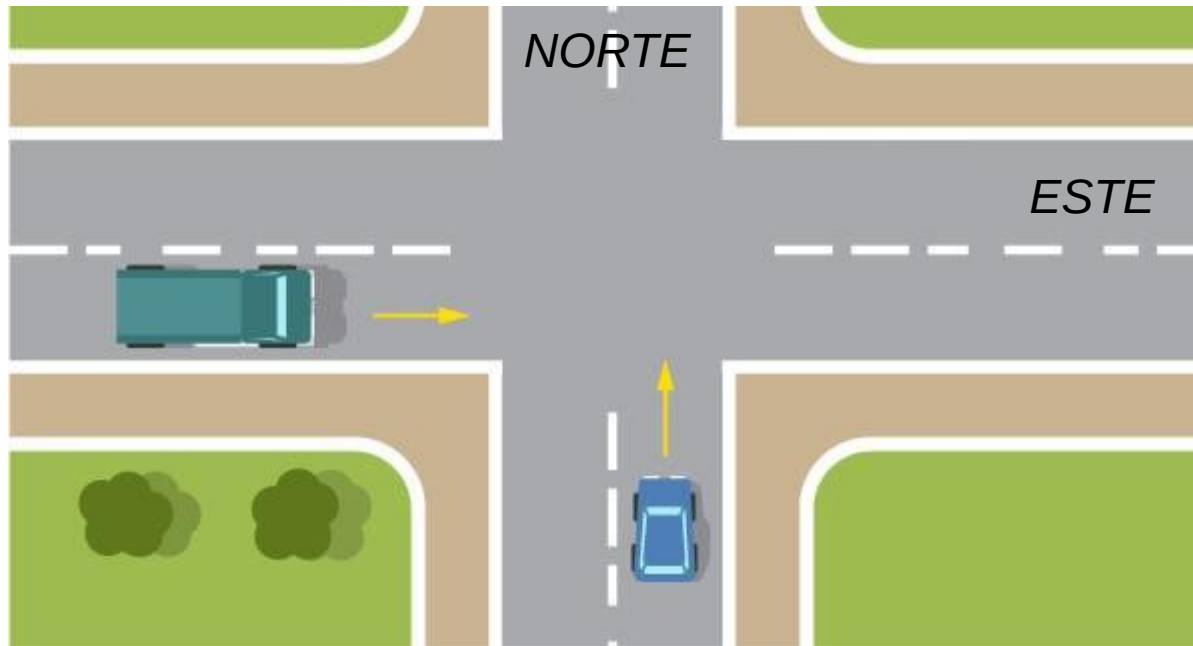
00:calles vacias
01:autos hacia el este
10:autos hacia el norte
11:autos en ambas dir.

Estados y salidas (Moore)

goN,Out0:verde hacia el norte & rojo hacia el este
waitN,Out1:amarillo hacia el norte & rojo hacia el este
goE,Out2:verde hacia el este & rojo hacia el norte
waitE,Out3:amarillo hacia el este & rojo hacia el norte

Ejemplo4: Control de tráfico

Variable Out



- B0: Verde hacia el norte*
- B1: Amarillo hacia el norte*
- B2: Rojo hacia el norte*
- B3: Verde hacia el este*
- B4: Amarillo hacia el este*
- B5: Rojo hacia el este*
- B6: X*
- B7: X*

salidas (Moore)

- Out0: verde hacia el norte & rojo hacia el este*
- Out1: amarillo hacia el norte & rojo hacia el este*
- Out2: verde hacia el este & rojo hacia el norte*
- Out3: amarillo hacia el este & rojo hacia el norte*

Por ejemplo Out0:

Out= 0b00100001

Ejemplo4: Control de tráfico

- Tabla de transición de estados:

Estados\Entradas	00	01	10	11
goN, Out0, 30seg	goN	waitN	goN	waitN
waitN, Out1, 5seg	goE	goE	goE	goE
goE, Out2, 30seg	goE	goE	waitE	waitE
waitE, Out3, 5seg	goN	goN	goN	goN

Notar que: las entradas digitales serán encuestadas luego de vencido el plazo de tiempo en un estado. Con lo cuál el cambio de estado será en función de la entrada pero solo cuando el evento tiempo (delay) se cumpla.

Este tipo de MEF no-periódica se llama Event-driven

Esto debe especificarse en diagrama de alguna forma (veremos más ejemplos).

Ejemplo4: Control de tráfico

- Implementación:

Estados\Entradas	00	01	10	11
goN, Out0, 30seg	goN	waitN	goN	waitN
waitN, Out1, 5seg	goE	goE	goE	goE
goE, Out2, 30seg	goE	goE	waitE	waitE
waitE, Out3, 5seg	goN	goN	goN	goN

```
//Estados posibles
typedef enum {goN,waitN,goE,waitE} state_name;
//estructura de un estado (1 fila de la tabla)
typedef struct state_row {uint8_t Out;
                          uint16_t Time;
                          state_name Next[4];
};

//Tabla completa
state_row MEF[4]={
    {Out0,30,{goN,waitN,goN,waitN}},
    {Out1, 5,{goE,goE,goE,goE}},
    {Out2,30,{goE,goE,waitE,waitE}},
    {Out3, 5,{goN,goN,goN,goN}}
};
```

*Definición y
Declaración de
Variables de estado*

*Definición de la MEF
la tabla es un arreglo de estructuras y no
un matriz de elementos del mismo tipo.*

```
main()
{
    uint8_t inputs;
    state_name state;
```

*Ejecución de la MEF
Método de Inicialización*

```
    Init_MEF();
    while(1){
        inputs=readSensors();
        state=MEF[state].Next[inputs];
        LightsUpdate(MEF[state].Out);
        Delay(MEF[state].Time);
    }
```

*No tiene temporización periódica o CLK
Los delay son distintos para cada estado.
Por lo tanto es arquitectura event-driven*

Más ejemplos

- Veremos en la próxima clase más ejemplos de sistemas modelados con MEF y su implementación en Microcontroladores

Bibliografía

- *“Real-Time Systems Design and Analysis”*. P. Laplante 3rd Ed. John Wiley & Sons 2004.
- *“Embedded Microcomputer System, Real-Time Interfacing”*. J. Valvano 2nd Ed. Thomson 2007.
- *“Fundamentos de lógica digital con diseño VHDL -”* 2nd Ed. Brown, Vranesic. 2006.
- *“Patterns for time-triggered Embedded Systems”*, Michael J. Pont. 2014 (pttes_2014.pdf descarga gratis en la web)
- *“Practical UML Statecharts in C/C++, 2nd Edition: Event-Driven Programming for Embedded Systems”*, Miro Samek, Newnes 2008, pdf descarga gratis en la web: <https://www.state-machine.com/>